AI Usage Declaration — Week 06

1. Declaration

We used AI tools for this project during the reporting period.

25125049 - Trần Hồng Vy (TV1)

I used **ChatGPT (OpenAI)** to review the C++17 Core utilities (`DynamicArray` and `Date`), discuss shared data types and object ID conventions, and check the consistency of models and manager interfaces across modules. ChatGPT helped identify design issues involving abstract `Transaction` objects, memory ownership, and Statistics interfaces. It also provided guidance for Git branching, Pull Request review, CMake integration, compilation checks, smoke tests, and the weekly documentation.

I inspected and revised the suggested changes, edited code and documentation, and executed Git, compilation, and testing commands on my own computer. I reviewed and merged the team's Pull Requests through GitHub. AI assistance did not replace code verification, and unimplemented features were not treated as completed.

26125041 - Lê Thiện Khôi (TV2)

I used **Gemini (Google)** to understand the Sprint 1 requirements for Master Data & Wallets; plan the `Wallet`, `IncomeSource`, and `ExpenseCategory` models; and design fixed-size binary records with a separate Name Pool. Gemini assisted in drafting manager interfaces and possible `.cpp` skeletons, discussing wallet balance operations and cross-module dependencies, and explaining CMake and Git workflows on Windows.

I reviewed and adapted the AI-assisted material and performed development tasks on my own computer. The submitted Sprint 1 PR contains the model and manager **headers**; full CRUD implementations and binary file I/O are not claimed as completed.

26125089 - Dương Thị Nhã Uyên (TV3)

I used **Claude (Anthropic)** for step-by-step guidance on the Sprint 1 Transactions and Recurring Transactions tasks and the Git/GitHub workflow. Claude provided draft declarations and explanations for the six header files, including transaction attributes, recurrence rules, manager interfaces, and relationships with Wallets, Income Sources, and Expense Categories.

I reviewed and adjusted those drafts, including changing the monetary type to the team's agreed `long long`. I created and committed the files and ran the Git commands on my own computer, and I coordinated shared data types and interfaces with teammates directly.

26125070 - Trần Nhật Tân (TV4)

I submitted a declaration describing the use of **ChatGPT (OpenAI)** to interpret the initial Statistics and Dashboard requirements; prepare draft C++ interface skeletons, design documentation, and sample test cases; and obtain guidance on CMake, testing, and Git/GitHub. The submitted declaration states that I performed the standalone build check, committed the files, and opened the Pull Request myself.

---

2. AI Tool Information

25125049 - Trần Hồng Vy (TV1)

Item	Details
AI Tool	ChatGPT
Provider	OpenAI
Platform	ChatGPT
Model	GPT-6
Usage Period	07/11/2026 – 13/11/2026
Purposes	Core code review, shared types and interface alignment, Git/PR guidance, build and test commands, documentation.
Evidence	ChatGPT conversation history; `DynamicArray.h`, `Date.h`, `Date.cpp`, `tests/test_core.cpp`; integrated CMake changes; GitHub PRs #5–#8 and commit history.

26125041 - Lê Thiện Khôi (TV2)

Item	Details
AI Tool	Gemini
Provider	Google
Platform	Gemini Web App
Model	Gemini
Usage Period	07/11/2026 – 13/11/2026
Purposes	Master Data models, binary record and Name Pool design, manager interfaces, balance operations, CMake/Git guidance.
Evidence	Gemini conversation history; `include/models/`, `include/managers/`; `feat/master-data-models` commits and PR #7.

26125089 - Dương Thị Nhã Uyên (TV3)

Item	Details
AI Tool	Claude
Provider	Anthropic
Platform	Claude (web/mobile chat)
Model	Claude Sonnet 5.5
Usage Period	07/11/2026 – 13/11/2026
Purposes	Git/GitHub guidance, transaction/recurring header design and explanations, team coordination, troubleshooting.
Evidence	Claude conversation history; `feat/transaction-models` commit history and PR #8.

26125070 - Trần Nhật Tân (TV4)

Item	Details
AI Tool	ChatGPT
Provider	OpenAI
Platform	ChatGPT
Model	GPT-6
Usage Period	07/11/2026 – 13/11/2026
Purposes	Statistics/Dashboard design, draft skeletons and test cases, build and Git guidance.
Evidence	ChatGPT conversation history (as reported); standalone build output; `feat/statistics-design` and PR #5.

---

3. Prompt Log

25125049 - Trần Hồng Vy (TV1)

- **AI Tool**: ChatGPT (OpenAI)
- **Purpose 1: Review and test Core utilities.**
    - **Prompt 1 (summary):** Review my `DynamicArray` and `Date` implementations carefully; check whether they are correct and ready for a Pull Request.
    - **Prompt 2 (summary):** Do not approve my PR just because I am the team leader; review every changed file and identify any real issues.
    - **Prompt 3 (summary):** Help prepare the Core Pull Request title, description, test checklist, and issue reference.

- **Purpose 2: Establish shared types and coding consistency.**
    - **Prompt 4 (summary):** Should we use floating-point or integer types for money in a VND-based finance application?
    - **Prompt 5 (summary):** Can `long long` be used for object IDs, and would `long long` for money with `int` for IDs be sufficient?
    - **Prompt 6 (summary):** We do not need a separate `Types.h`; help communicate the agreed types and `Date` interface to the team.
    - **Prompt 7 (summary):** Explain `#pragma once` versus `#ifndef` and help standardize matching model and manager interfaces without removing required methods.
- **Purpose 3: Review and integrate teammates' modules.**
    - **Prompt 8 (summary):** Review the Master Data and Transactions Pull Requests step by step, including naming conventions, comments, constructors, ID handling, and memory ownership.
    - **Prompt 9 (summary):** The Statistics module uses `double` and `DynamicArray<Transaction>`. Help align it with `long long` and the abstract `Transaction` design.
    - **Prompt 10 (summary):** Check the documentation, source files, and CMake integration before merging the Statistics/Dashboard Pull Request.
- **Purpose 4: Git troubleshooting, verification, and reporting.**
    - **Prompt 11 (summary):** Explain how to switch to teammates' feature branches, commit review fixes, and push updates without including unrelated files.
    - **Prompt 12 (summary):** Help resolve Git's `vi` editor error during a merge while preserving uncommitted evidence files.
    - **Prompt 13 (summary):** Suggest C++17 header compilation checks, linking and smoke tests, and integrated CMake build commands.
    - **Prompt 14 (summary):** Prepare the Week 06 Weekly Report based on actual Sprint 1 progress and distinguish completed interfaces from unimplemented features.

26125041 — Lê Thiện Khôi (TV2)

- **AI Tool:** Gemini (Google)
- **Purpose 1: Analyze the Sprint 1 requirements and design Master Data models.**
    - **Prompt 1:** “Here is our team's Week 1 task assignment for the C++17 Personal Finance Manager project. Help me break down the deliverables and requirements for TV2 (Master Data & Wallets).”
    - **Prompt 2:** “Design the attributes, validation rules, and soft-deletion flags for the Wallet, IncomeSource, and ExpenseCategory classes so they work seamlessly with a custom DynamicArray container.”
    - **Prompt 3:** “What currency data type should we use for Wallet balances in C++17 to avoid floating-point precision issues while remaining easy to adjust if the team updates conventions?”
- **Purpose 2: Design binary records and Name Pool structures.**
    - **Prompt 4:** “Explain how to separate persistent Object IDs from dynamic Record Indexes when storing Master Data objects in a custom DynamicArray and binary files.”
    - **Prompt 5:** “Since std::string cannot be written directly to binary files, design trivially copyable fixed-size record structs (WalletRecord, IncomeSourceRecord, ExpenseCategoryRecord) using a Name Pool with byte offset and length.”
    - **Prompt 6:** “Write the toRecord() and fromRecord() serialization helper methods for Wallet, IncomeSource, and ExpenseCategory, including compile-time static_assert checks.”

- **Purpose 3: Plan Manager interfaces and cross-module integration.**
    - **Prompt 7:** “Design the header interfaces for WalletManager, IncomeSourceManager, and ExpenseCategoryManager supporting CRUD operations, ID/name lookups, and binary persistence hooks.”
    - **Prompt 8:** “Define atomic wallet balance update methods (deposit, withdraw, canWithdraw, adjustBalance, and getTotalBalance) so the Transactions and Statistics modules can safely interact with WalletManager.”
    - **Prompt 9:** “Generate clean C++17 .cpp skeleton implementations for all three managers that compile without unused-parameter warnings under -Wall -Wextra before DynamicArray methods are finalized.”
- **Purpose 4: Build and Git workflow guidance.**
    - **Prompt 10:** “Help me verify the CMake build setup using MSYS2 MinGW-w64 on Windows and outline the Git branch and commit steps for pushing feat/master-data-models.”

26125089 — Dương Thị Nhã Uyên (TV3)

- **AI Tool:** Claude (Anthropic)
- **Purpose 1: Understand the Sprint 1 tasks and learn Git/GitHub.**
    - **Prompt 1:** “This is everything I have to do this week (transaction and recurring transaction models, manager interfaces, team coordination). Guide me step by step, because I have never used GitHub.”
    - **Prompt 2:** “Here are plan_for_week_1.md and CODING_RULES.md.” (The two files were supplied to guide the response.)
    - **Prompt 3:** “What does git checkout -b feat/transaction-models mean, and how do I check that the branch was created?”
    - **Prompt 4:** “What am I doing with git add and git commit?”
- **Purpose 2: Design and understand models and manager interfaces.**
    - **Prompt 5:** “I copied your code into Transaction.h but I don’t understand it. Please explain it.”
    - **Prompt 6:** “Define transaction attributes: ID, Date, Amount, Description, Wallet ID, and Income Source ID or Expense Category ID. I have not done this yet. Where is it in my code?”
    - **Prompt 7:** “Recurring Transactions: define the type, Start Date and optional End Date, monthly recurrence rules, and fields to prevent duplicates. Which part of my code covers these? I think I did not declare the type.”
    - **Prompt 8:** “I have not created the manager files yet. Which operations (Add, View, Edit, Delete) must I declare?”
- **Purpose 3: Coordinate types and check acceptance criteria.**
    - **Prompt 9:** “How do I agree on the types with other members, and do I have to change my code after we agree?”
    - **Prompt 10:** “What is the status of each Acceptance Criterion, and what is left to do?”
    - **Prompt 11:** “Our team uses long long for money.”
    - **Prompt 12:** “What is int64_t?”
- **Purpose 4: Troubleshoot and submit work.**
    - **Prompt 13:** “Why is there a red squiggle under the #include lines for models/Transaction.h and core/Date.h?”
    - **Prompt 14:** “TV1 ticked Date and DynamicArray as done, but I cannot find Date.h. Why?”
    - **Prompt 15:** “What does this mean?” (Screenshot of `git status` with modified files and an unwanted shortcut file.)
    - **Prompt 16:** “What should I do next?” (Screenshot of a successful `git push`.)

26125070 — Trần Nhật Tân (TV4)

- **AI Tool:** ChatGPT (OpenAI)
- **Purpose 1: Clarify Sprint 1 responsibilities.**
    - **Prompt 1 (English summary supplied by TV4):** "What should I deliver in Week 1 as TV4 under the team plan?"
- **Purpose 2: Prepare Statistics/Dashboard deliverables.**
    - **Prompt 2 (English summary supplied by TV4):** "Help prepare the Week 1 class skeletons, design documentation, and sample test cases using the team structure."
- **Purpose 3: Build and submit the skeleton.**
    - **Prompt 3 (English summary supplied by TV4):** "Guide me through building with CMake on Windows, splitting the work into commits, and opening a pull request for TV1."